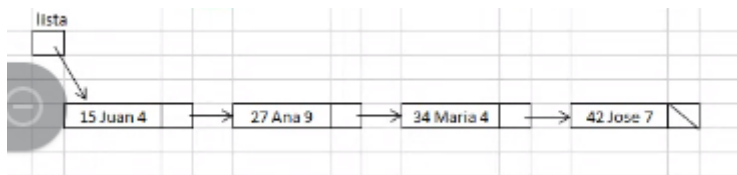


Algoritmos y Estructuras de Datos

Listas

Listas simples

En este ejemplo tenemos una lista de listas con matrícula, nombre y nota. Definimos un struct Alumno con legajo, nombre y nota y un struct Nodo con info y puntero sig. La lista está ordenada, y la idea es que siempre se mantenga ordenada, que la trabajemos ordenada. Si insertamos algo debe ser de forma ordenada.



```
struct Alumno
{
    unsigned leg;
    char nom[31];
    unsigned nota;
};

struct Nodo
{
    Alumno info;
    Nodo*sig;
};
```

Mostrar en lista

Hacemos una función para mostrar todo lo que hay en la lista. La función listar recibe la lista que es el puntero al Nodo. No la pasamos por referencia porque no se va a modificar, solo se va a mostrar.

Voy a hacer un ciclo while que dure mientras no termine de mostrar a todos los alumnos (o sea cuando r sea distinto de NULL). Antes definimos una variable de tipo puntero r inicializada en lista (ahí es donde está el primer elemento de la lista) que va a funcionar como el i de un vector cuando se lo recorre. Dentro del ciclo voy a hacer un cout que muestre r->info.leg y r->info.nom y r->info.nota. Ahora hacemos r=r->sig par que apunte al siguiente.

```

void listar(Nodo*lista)
{
    Nodo*r;
    r=lista;
    while (r!=NULL)
    {
        cout<<r->info.leg<<r->info.nom<<r->info.
        r=r->sig;
    }
}

```

Acá podría haberme evitado de usar la variable r y usar directamente la variable lista.

Esta función funciona con una lista vacía.

Ahora vamos a hacer una función void listarAprobados que recibe la lista pasada por valor. Ahora solo vamos a mostrar los alumnos aprobados.

```

void mostrarAprobados(Nodo*lista)
{
    Nodo*r;
    r=lista;
    while (r!=NULL)
    {
        if (r->info.nota>=6)
            cout<<r->info.leg<<r->info.nom<<r->info.nota<<endl;
        r=r->sig;
    }
}

```

Vamos a hacer una función float promedio que recibe la lista pasada por valor que devuelva el promedio de la nota de todos los nodos.

```

float promedioCurso(Nodo*lista)
{
    unsigned cont=0, suma=0;
    Nodo*r;
    r=lista;
    while (r!=NULL)
    {
        suma+=r->info.nota;
        cont++;
        r=r->sig;
    }
    if (cont>0)
        return (float) suma/cont;
    else
        return 0;
}

```

Búsqueda secuencial en lista

La función buscar la podemos usar, por ejemplo, si nos dan el número de legajo de un alumno y queremos mostrar qué nota tiene o para modificarle la nota.

Lo que me va a devolver la función de búsqueda es un puntero al nodo que estaba buscando. Va a recibir el puntero a la lista y el elemento que estoy buscando.

Definimos la función `Nodo* buscar(Nodo* lista, unsigned unLeg)`

Dentro de la función declaramos e inicializamos una variable `r` en el primer elemento y hacemos un ciclo `while` que se cumpla mientras no termine de recorrer la lista y mientras que donde esté parada no sea lo que estoy buscando. En caso de entrar al `while`, avanzo en la lista.

Afuera del ciclo `while` retornamos `r`, que va a tener la información del elemento buscado. En caso de no encontrar el elemento va a retornar `NULL`.

```
Nodo* buscar(Nodo* lista, unsigned unLeg)
{
    Nodo* r = lista;
    while (r != NULL && r->info.leg != unLeg)
        r = r->sig;
    return r;
}
```

Se puede hacer una mejora a esta función, en el caso de que si lo que busco no esté en la lista. Por ejemplo, si yo busco el número 30, pero en un momento dado llega al recorrer la lista llega al 34, quiere decir que el 30 no está y ya no es necesario recorrer más, puesto que la lista está ordenada y la búsqueda es secuencial.

La mejora consiste en agregarle dentro del `while` la condición de que la información del nodo `r` sea menor al elemento que se está buscando. En este caso, además debo agregar un `if` afuera del `while` preguntando si `r` es distinto a `NULL` y si `r` es igual al elemento que se busca. Si ocurre eso se devuelve `r` y si no se devuelve `NULL`.

```
Nodo* buscarMejor(Nodo* lista, unsigned unLeg)
{
    Nodo* r = lista;
    while (r != NULL && r->info.leg < unLeg)
        r = r->sig;
    if (r != NULL && r->info.leg == unLeg)
        return r;
    else
        return NULL;
}
```

Modificar en lista

Vamos a hacer una función que, dado un legajo, modifique la nota que se encuentra en la información del nodo que corresponde. Esta función recibe el nodo lista, el legajo, y la nueva nota.

Definimos una variable `Nodo *p` donde se va a guardar el resultado de la función `buscar`. Luego pregunto si `p` es distinto de `NULL`. Si es el caso modifico la nota. Como no cambió el orden de los nodos, da igual pasarlo por referencia o no.

```

void modificarNota (Nodo*lista, unsigned unLeg, unsigned nuevaNota)
{
    Nodo*p=buscarMejor (lista, unLeg);
    if (p!=NULL)
        p->info.nota=nuevaNota;
}

```

Insertar en lista

La idea de la función insertar es agregar un nodo en cualquier lugar, de forma ordenada. Esta función va a recibir la lista y un alumno.

Defino una variable `Nodo n`, a la que le voy a poner un nuevo nodo (`n = new Nodo`). Luego lleno la información de `n` poniendo `n->info=alu`. Este nodo está suelto ahora mismo. Para poder insertarlo en la lista voy a necesitar la dirección de memoria del nodo sucesor (para hacer que el sig del nuevo nodo apunte a ese) y la del nodo antecesor (para hacer que su sig apunte al nuevo nodo).

Cuando insertamos un nodo en una lista tenemos 4 posibles casos:

- Que insertemos un nodo que va a tener antecesor y sucesor.
- Que insertemos un nodo que va a tener sucesor pero no antecesor (primera posición).
- Que insertemos un nodo que va a tener antecesor pero no sucesor (última posición).
- Que insertemos un nodo que no va a tener ni antecesor ni sucesor (insertar en lista vacía).

La función que hagamos debe funcionar para estos 4 casos.

Probemos primero insertando un **nodo intermedio**:

Lo primero que debo hacer es encontrar al sucesor del nuevo nodo, que es el primer mayor a ese nuevo. Voy a declarar, además de `n`, una variable `r` inicializada en el primero (en lista). Luego voy a hacer un ciclo `while` que se cumpla mientras `r->info.leg` sea menor a `alu.leg`. De este ciclo voy a salir apuntando a quién va a ser el sucesor de `n`. Dentro del ciclo le asigno a `r=r->sig`.

También necesitamos el antecesor. Defino una variable `ant` y dentro del ciclo `while`, antes de mover al siguiente, le asigno `ant=r`.

Una vez que salgo del ciclo `while` hago `n->sig=r`, es decir, al siguiente de `n` le asigno el `r`, el sucesor y al anterior le asigno `n`, `ant->sig=n`.

```

void insertar(Nodo*lista,Alumno alu)
{
    Nodo *n,*r,*ant;
    n=new Nodo;
    n->info=alu;
    r=lista;
    while(r->info.leg<alu.leg)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    ant->sig=n;
}

```

Probemos ahora insertando un **último nodo**:

En este caso es igual al anterior, salvo que lo que debemos hacer es agregarle una condición en el while, que dice que r sea distinto de NULL. Si no hiciera esto, al llegar al último r me quedaría en NULL, y tendría un NULL antes de una flecha.

```

void insertar(Nodo*lista,Alumno alu)
{
    Nodo *n,*r,*ant;
    n=new Nodo;
    n->info=alu;
    r=lista;
    while(r!=NULL && r->info.leg<alu.leg)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    ant->sig=n;
}

```

Probemos ahora insertando un **primer nodo**:

Acá también es igual que el anterior, salvo que en este caso no va a entrar al ciclo. Entonces, afuera del while debo preguntar si entré al ciclo o no. Si nunca entré al ciclo r y lista son iguales. Entonces en la condición voy a preguntar si r es distinto de lista, en este caso no estaría insertando un primer nodo y la situación queda igual que la anterior, con el ant->sig=n. En el else, lo que debo modificar es lista, pidiéndole que sea igual a n. **Aclaración:** lista debe estar pasado por referencia porque lo estoy modificando.

```

void insertar(Nodo*&lista, Alumno alu)
{
    Nodo *n, *r, *ant;
    n=new Nodo;
    n->info=alu;
    r=lista;
    while(r!=NULL && r->info.leg<alu.leg)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    if(r!=lista)
        ant->sig=n;
    else
        lista=n;
}

```

Por último, vamos al caso de insertar un nodo en una **lista vacía**:

Acá la función anterior sirve también. Entonces, la función final completa nos queda así:

```

void insertar(Nodo*&lista, Alumno alu)
{
    Nodo *n, *r, *ant;
    n=new Nodo;
    n->info=alu;
    r=lista;
    while(r!=NULL && r->info.leg<alu.leg)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    if(r!=lista)
        ant->sig=n;
    else
        lista=n;
}

```

Crear una lista desde 0

Si quiero crear una lista puedo usar la función insertar, y en el main quedaría algo así:

```

int main()
{
    Nodo*listaK1021=NULL;
    Alumno a;
    cin>>a.leg;
    while (a.leg!=0)
    {
        cin>>a.nom;
        cin>>a.nota;
        insertar(listaK1021,a);
        cin>>a.leg;
    }
    listar(listaK1021)
}

```

Función buscarInsertar

Esta función hace lo mismo que hace la función buscar, con la diferencia de que si no encuentra el elemento que se busca, lo inserta. La misma devuelve el puntero al nodo.

Definimos la función `Nodo * buscarInsertar`, que recibe la lista por referencia y el alumno.

Defino las variables `r` y `ant` y luego a `r` le asigno lista. Posteriormente pongo el mismo ciclo `while` de la función `insertar`.

Una vez que salgo del ciclo pregunto si el elemento que estaba buscando no está, que es el caso de que `r` sea `NULL` o que `r->info.leg` sea distinto al legajo que se busca. Si este es el caso, entonces tengo que agregarlo. Para ello voy a pedir memoria declarando un nodo `n` y poniendo `new Nodo`. Completo la info de `n` con `alu` y luego lo enlazo tal como en la función `insertar`. En este caso la función devuelve el puntero al nodo ya sea porque lo encontró o lo agregó, es decir, devuelve `r`. En caso de que el elemento sí estaba desde el comienzo, entonces simplemente devuelvo `r`.

```

Nodo*buscarInsertar(Nodo*&lista,Alumno alu)
{
    Nodo*r,*ant;
    r=lista;
    while(r!=NULL && r->info.leg<alu.leg)
    {
        ant=r;
        r=r->sig;
    }
    if(r==NULL || r->info.leg!=alu.leg)
    {
        Nodo*n=new Nodo;
        n->info=alu;
        n->sig=r;
        if(r!=lista)
            ant->sig=n;
        else
            lista=n;
        return n;
    }
    else
        return r;
}

```

Función eliminar

Esta función recibe la lista pasada por referencia y la clave del elemento que quiero eliminar. Primero debería buscar quién es ese elemento que quiero eliminar, es decir, debo buscar el nodo. Además, cuando elimine el sig del anterior tiene que apuntar al posterior del que se eliminó. Entonces, no solo necesito el nodo a eliminar sino también el antecesor y el sucesor, si los hubiera.

Cuando eliminamos un nodo en una lista tenemos 4 posibles casos:

- Que eliminemos un nodo que va a tener antecesor y sucesor.
- Que eliminemos un nodo que va a tener sucesor pero no antecesor (primera posición).
- Que eliminemos un nodo que va a tener antecesor pero no sucesor (última posición).
- Que eliminemos un nodo que no va a tener ni antecesor ni sucesor (insertar en lista vacía).

La función que hagamos debe funcionar para estos 4 casos.

Copiamos el ciclo while de buscarInsertar. Cuando salgamos del ciclo, si el elemento estaba entonces r va a estar apuntando a lo que yo estaba buscando. Copiamos el if de buscarInsertar por si no lo llega a encontrar, y en ese caso ponemos un mensaje de que no está.

Si encontré el elemento, entonces tengo que desengancharlo y borrarlo. Para hacer esto, al antecesor del elemento a borrar lo tengo que enlazar con el sucesor. Entonces ant->sig = r->sig y después hago un delete r.

Si fuera el caso de que elimino el primero, entonces no tendría anterior. Por ende antes de eliminar debo preguntar si r es igual a lista. En caso de serlo, asigno lista = r->sig, puesto que le estoy asignando el sucesor del primer elemento. En caso de no ser igual, en el else se pone lo que está en el párrafo anterior.

```
void eliminar(Nodo*&lista, unsigned unLeg)
{
    Nodo*r, *ant;
    r=lista;
    while(r!=NULL && r->info.leg<unLeg)
    {
        ant=r;
        r=r->sig;
    }
    if(r==NULL || r->info.leg!=unLeg)
        cout<<"no esta ";
    else
    {
        if(r==lista)
            lista=r->sig;
        else
            ant->sig=r->sig;
        delete r;
    }
}
```

También esta función podría ser booleana:


```

bool eliminar(Nodo*&lista, unsigned unLeg)
{
    Nodo*r,*ant;
    r=lista;
    while(r!=NULL && r->info.leg<unLeg)
    {
        ant=r;
        r=r->sig;
    }
    if(r==NULL || r->info.leg!=unLeg)
        //cout<<"no esta ";
        return false;
    else
    {
        if(r==lista)
            lista=r->sig;
        else
            ant->sig=r->sig;
        delete r;
        return true;
    }
}

```

Apareo en listas

Tenemos dos listas de alumnos ordenadas por legajo y queremos mostrarlos a todos juntos ordenados en una sola lista y, si se repite algún alumno en las dos listas, se informa la mayor nota.

Hacemos una función apareo que recibe las dos listas. El apareo consiste en recorrer las dos estructuras al mismo tiempo mientras no haya terminado de recorrer ninguna de las dos.

Para recorrer las dos estructuras defino dos punteros, p1 (lista 1) y p2 (lista 2). Inicializo ambos punteros con sus respectivas listas con el signo =.

Hacemos un ciclo while con la condición de que p1 y p2 sean distintos de NULL, es decir, que se cumpla mientras no termine de recorrer ninguna de las dos listas. Ahora como en todo apareo, se hacen los tres posibles casos:

- Si la info de p1 es menor que la info de p2, se procesa la info de p1, es decir, se muestra, y luego se avanza en p1.
- Si la info de p1 es igual que la info de p2, se muestra cualquiera de los dos y se avanza en los dos. En este caso, también agregamos la condición de que si la nota de uno de los dos es mayor, que se muestre ese.
- Si la info de p1 es mayor que la info de p2, se procesa la info de p2 y se avanza en p2.

Cuando se sale del ciclo while, debemos procesar todo lo que nos queda pendiente en cualquiera de las dos listas, si se da el caso. Entonces hacemos dos ciclos while, uno que muestre y recorra p1 y otro que muestre y recorra p2.

```
void apareo(Nodo*lista1, Nodo*lista2)
{
    Nodo *p1,*p2;
    p1=lista1;
    p2=lista2;
    while (p1!=NULL && p2!=NULL)
    {
        if (p1->info.leg<p2->info.leg)
        {
            cout<<p1->info.leg<<p1->info.nom<<p1->info.nota<<endl;
            p1=p1->sig;
        }
        else
        {
            if (p1->info.leg==p2->info.leg)
            {
                cout<<p1->info.leg<<p1->info.nom;
                if (p1->info.nota>p2->info.nota)
                    cout<<p1->info.nota<<endl;
                else
                    cout<<p2->info.nota<<endl;
                p1=p1->sig;
                p2=p2->sig;
            }
            else
            {
                cout<<p2->info.leg<<p2->info.nom<<p2->info.nota<<endl;
                p2=p2->sig;
            }
        }
    }
}
```

```
while (p1!=NULL)
{
    cout<<p1->info.leg<<p1->info.nom<<p1->info.nota<<endl;
    p1=p1->sig;
}
while (p2!=NULL)
{
    cout<<p2->info.leg<<p2->info.nom<<p2->info.nota<<endl;
    p2=p2->sig;
}
}
```

Corte de control en listas

En este caso tenemos una lista con alumnos repetidos y queremos mostrar los alumnos y su promedio.

Vamos a hacer dos formas diferentes:

Forma 1

Esta es una forma más genérica. La condición de un corte de control es que los datos tienen que estar agrupados por uno de sus miembros, es decir, todos los repetidos tienen que estar juntos. Vamos a tener dos ciclos while, uno que va a iterar los alumnos y otros que va a ir por repetidos.

Definimos una variable p inicializada en lista y declaramos una variable legA que va a ser inicializada dentro del ciclo cada vez que se empiece con un nuevo alumno. Además, para poder sacar el promedio necesito una variable suma y contador que se van a inicializar en 0 dentro del while.

Empezamos el ciclo while con la condición de que p sea distinto de NULL. Le asignamos a la variable legA el legajo de p. Inicializamos suma y contador en 0. El segundo ciclo es un do while que se va a cumplir mientras p sea distinto de NULL y el legajo de p es igual a legA. En este ciclo vamos a ir sumando las notas de todos los alumnos y además sumamos 1 al contador. Luego pasamos al siguiente alumno.

Una vez que termina un alumno, mostramos su legajo y el promedio. Si quisiera guardar el nombre, debería usar una variable como legA pero para guardar el nombre.

```
void cc(Nodo*lista)
{
    Nodo *p=lista;
    int legA,suma,cont;
    string nomA;
    while (p!=NULL)
    {
        legA=p->info.leg;
        nomA=p->info.nom;
        suma=cont=0;
        do
        {
            suma+=p->info.nota;
            cont++;
            p=p->sig;
        }while (p!=NULL && p->info.leg==legA);
        cout<<legA<<nomA<<<suma/cont<<endl;
    }
}
```

Forma 2

En esta forma en vez de guardar el legajo, vamos a trabajar con dos punteros. Uno va a apuntar al primer nodo de cada grupo y otro puntero que va a ir iterando por los que son iguales. Entonces, en vez de usar el legA, dejo al puntero apuntando al primer nodo de la lista.

Defino dos punteros p y q, inicializo p en lista. Además, para poder sacar el promedio necesito una variable suma y contador que se van a inicializar en 0 dentro del while.

Hago un ciclo while mientras p sea distinto de NULL. Lo primero que ocurre en este ciclo es que q es igual a p, ya que q va a recorrer todos los elementos repetidos, incluido el primero. Asimismo, inicializamos suma y contador en 0.

Después hago un ciclo do while mientras que q sea distinto de NULL y el legajo de p sea igual que el legajo de q, en donde sumemos la nota de lo que está contenido en q, aumentamos el contador y nos movemos en q.

Una vez que termina un alumno, mostramos su legajo y el promedio procesando p. Si quisiera guardar el nombre, debería usar una variable como legA pero para guardar el nombre, en este caso, nomA.

Finalmente, a p le asigno q para que arranque con el próximo alumno.

```
void cc(Nodo*lista) //versión con 2 punteros
{
    Nodo*p,*q;
    int suma,cont;
    p=q=lista;
    while(p!=NULL)
    {
        //q=p;
        suma=cont=0;
        do
        {
            suma+=q->info.nota;
            cont++;
            q=q->sig;
        }while(q!=NULL && p->info.leg==q->info.leg);
        cout<<p->info.leg<<p->info.nom<<suma/cont<<endl;
        p=q;
    }
}
```

Ordenar una lista

Forma 1

Hacemos una función ordenar que va a recibir la lista por referencia (porque se modifica). En esta versión vamos a crear una nueva lista ordenada. Lo que voy a hacer es ir recorriendo la lista, eliminando los nodos, y esos datos insertarlos en la nueva lista.

Definimos una lista auxiliar inicializada en NULL y una variable p.

Hacemos un ciclo while mientras lista sea distinto de NULL. Dentro del ciclo llamamos a la función insertar para insertar los datos en una lista vacía. Ahora debemos eliminar el dato que acabamos de pasar a la otra lista. Para eso a la variable p le asignamos lista, nuevo lista al siguiente elemento y borro p con delete.

Por último, hacemos que lista sea igual a lista auxiliar, pues lista auxiliar es una variable local. Con esto hacemos que lista apunte al primer elemento de la lista auxiliar, que está ordenada.

```
void ordenarl (Nodo*&lista)
{
    Nodo*listaAux=NULL;
    Nodo*p;
    while (lista!=NULL)
    {
        insertar (listaAux,lista->info);
        p=lista;
        lista=lista->sig;
        delete p;
    }
    lista=listaAux;
}

void insertar (Nodo*&lista,int dato)
{
    Nodo *n,*r,*ant;
    n=new Nodo;
    n->info=dato;
    r=lista;
    while (r!=NULL && r->info<dato)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    if (r!=lista)
        ant->sig=n;
    else
        lista=n;
}
```

Forma 2

La forma 1 no tiene demasiado sentido, puesto que se pide y se elimina memoria constantemente. En cambio, se puede trabajar con los mismos nodos. Simplemente se puede cambiar el orden de la lista modificando los punteros, no cambiando los nodos de lugar en la memoria. Puedo dejar los nodos en el mismo lugar y sólo cambiar los enlaces para que se enlacen de forma ordenada (los enlaces de los nodos es lo que les da el ordenamiento).

Vamos a trabajar igual que en la forma 1, pero con la diferencia de que la función insertar va a ser un insertar lógico, que no va a pedir memoria, sino que va a modificar los enlaces.

La función insertarLog recibe la lista por referencia y el puntero al nuevo nodo. Lo que va a hacer es buscar el lugar donde debe estar y lo engancha.

La función ordenar2 va a ser igual que la primera, salvo que lo que voy a hacer es tomar cada nodo, desengancharlo y se lo mando al insertarLog.

En el ciclo while de esta función guardamos en una variable p lista y luego sacamos los elementos lógicamente, modificando lista al asignarle el siguiente. O sea p va a apuntar al elemento que vamos a modificar y lista va a apuntar al siguiente elemento de ese. Ahora vamos a insertar el elemento que está apuntando p con el insertarLog, en una lista auxiliar.

```
void ordenar2 (Nodo*&lista)
{
    Nodo*listaAux=NULL;
    Nodo*p;
    while (lista!=NULL)
    {
        p=lista;
        lista=lista->sig;
        insertarLog (listaAux,p);
    }
    lista=listaAux;
}

void insertarLog (Nodo*&lista,Nodo*n)
{
    Nodo *r,*ant;

    r=lista;
    while (r!=NULL && r->info<n->info)
    {
        ant=r;
        r=r->sig;
    }
    n->sig=r;
    if(r!=lista)
        ant->sig=n;
    else
        lista=n;
}
```

Generar una lista desordenada

Para generar una lista desordenada, cada elemento que ingreso lo puedo ingresar atrás o adelante, similar a una cola.

Entonces hacemos una función agregarAdelante que recibe la lista por referencia y un dato. Pedimos memoria con una variable p, a la info de p le ponemos el dato y al siguiente de p le ponemos lista para que el anterior apunte al siguiente (que era el que estaba inicialmente) y luego a lista le asignamos p para que apunte a ese nuevo dato.

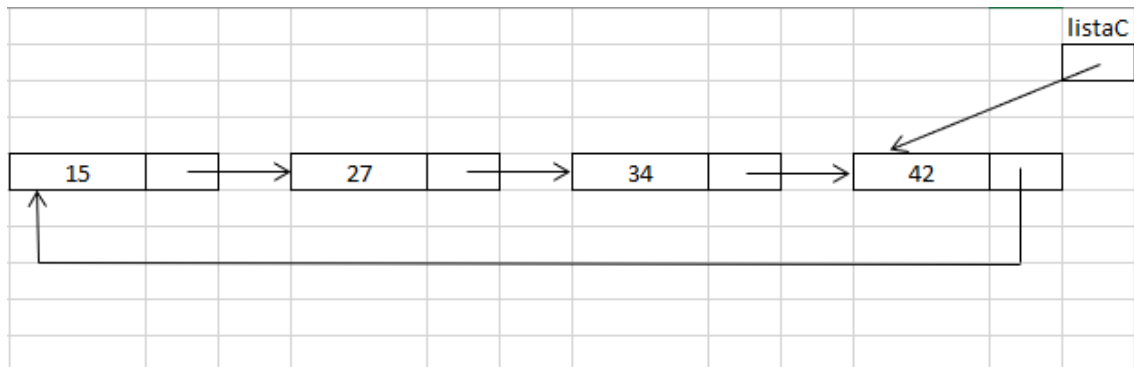
```
void agregarAdelante (Nodo*&lis,int nro)
{
    Nodo*p=new Nodo;
    p->info=nro;
    p->sig=lis;
    lis=p;
}
```

Listas circulares

Es una lista simplemente enlazada circular, donde el último elemento en lugar de tener NULL apunta al primero.

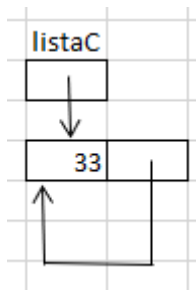
Si la lista está ordenada, el puntero que accede a la lista va a apuntar al mayor, así el primero de la lista es el siguiente.

Si la lista está desordenada, el puntero de acceso apunta a cualquier elemento.



En el caso de la lista vacía vamos a tener el puntero en NULL.

En el caso de que haya un solo elemento, ese elemento se apunta a si mismo.



Función mostrar

Esta función muestra la lista circular.

Acá debemos tener en cuenta si la lista está vacía al comienzo de la función. Si no está vacía, es decir, si listaC es distinto de NULL, definimos un puntero p que apunta al siguiente de listaC, es decir, al primer elemento de la lista. Hago un ciclo do while en donde muestro los datos y luego avanzo uno en p. La condición de este ciclo es que p sea distinto a listaC->sig, es decir, que sea distinto al primer elemento. Solo cuando sea igual al primer elemento me indicará que ya terminé de recorrer la lista circular.

```

void mostrar(Nodo* listaC)
{
    if(listaC!=NULL)
    {
        Nodo* p=listaC->sig;
        do
        {
            cout<<p->dato<<endl;
            p=p->sig;
        }while (p!=listaC->sig);
    }
}

```

Función agregar

Esta función agrega un nodo a una lista circular desordenada. También se usa para crear una lista desde cero.

Lo primero que hacemos es definir una variable n y pedimos memoria. A la info o dato de n le asignamos cualquier dato que queramos agregar, en este caso un número.

Después pregunto si la lista está vacía. Si no lo está, a n->sig le pongo listaC->sig, o sea, engancho el elemento que agregué al primer elemento de la lista. Además, listaC->sig va a dejar de apuntar a ese primer elemento para apuntar al elemento que agregamos, o sea, le asigno n.

En el caso de que la lista estuviera vacía, a n->sig le asigno n, o sea, hago que se apunte a sí mismo y a listaC también le asigno n así apunta a ese nuevo valor que agregamos.

```

void agregar(Nodo*&listaC, int nro)
{
    //agrega un nodo a la lista circular sin orden
    Nodo*n=new Nodo;
    n->dato=nro;
    if(listaC!=NULL)
    {
        n->sig=listaC->sig;
        listaC->sig=n;
    }
    else
    {
        n->sig=n;
        listaC=n;
    }
}

```


Función insertar

Esta función agrega un nodo a una lista circular ordenada.

Lo primero que hacemos es definir una variable n y pedimos memoria. A la info o dato de n le asignamos cualquier dato que queramos agregar, en este caso un número.

Luego pregunto si la lista está vacía. Si lo está, al siguiente de n le ponemos n y listaC apunta a n también.

En cambio, si la lista no está vacía, defino una variable p (sucesor) a la que le asigno el siguiente de listaC y una variable ant (antecesor) que inicializamos en listaC.

Luego pregunto si el dato de p es menor al dato que se quiere insertar, en este caso un número. Si es el caso, entonces hago un ciclo do while, donde a ant le asigno p y a p le asigno el siguiente de p. El ciclo va a hacerse mientras p sea distinto del siguiente de lista y el dato de p sea menor al dato que quiero insertar. Esto va a lograr que el antecesor y el sucesor me queden listos para poder ingresar el nuevo dato. Cuando termina el ciclo quiere decir que ya se dio una vuelta entera a toda la lista.

Una vez que termina el ciclo, al siguiente de n le asigno p y al siguiente del anterior le asigno n.

Finalmente pregunto si el dato de n es mayor al dato de listaC. Si es el caso, a listaC le asigno n y termino cambiando el puntero de listaC (recordemos que este siempre apuntaba al último elemento de la lista).

```
void insertar(Nodo*&listaC, int nro)
{
    //agrega un nodo a la lista circular ordenada
    Nodo*n=new Nodo;
    n->dato=nro;
    if(listaC==NULL)
    {
        n->sig=n;
        listaC=n;
    }
    else
    {
        Nodo*p=listaC->sig, *ant=listaC;
        if(p->dato<nro)
        {
            do
            {
                ant=p;
                p=p->sig;
            }while (p!=listaC->sig && p->dato<nro);

            n->sig=p;
            ant->sig=n;
            if(n->dato>listaC->dato)
                listaC=n;
        }
    }
}
```

Listas doblemente enlazadas

En una lista doblemente enlazada, cada elemento apunta al anterior y al siguiente, o sea, cada nodo ahora tiene dos punteros, uno al siguiente y uno al anterior. Entonces cuando queremos insertar no tenemos que guardar el anterior porque ya lo tenemos y cual es el siguiente. Si la lista estuviese ordenada, tenemos dos punteros externos mediante los cuales podemos acceder a la lista: con uno accedemos a la lista ordenada de forma ascendente y con otro, de forma descendente.



El nodo, a diferencia del nodo de otros tipos de listas, tiene dos punteros: uno al siguiente y uno al anterior. El primer anterior va a tener NULL.

```
struct Nodo
{
    int dato;
    Nodo*sig;
    Nodo*ant;
};
```

Mostrar ascendente

Esta función recibe el puntero de acceso a lista ascendente.

Definimos una variable de recorrido p y le asignamos el valor que recibe, en este caso l.

Luego hacemos un ciclo while que se recorra mientras p sea distinto de NULL. Dentro del ciclo hacemos un cout mostrando p->dato y vamos moviendo p.

```
void mostrarAsc (Nodo*l)
{
    Nodo*p=l;
    while (p!=NULL)
    {
        cout<<p->dato<<endl;
        p=p->sig;
    }
}
```

Mostrar descendente

Esta función recibe el puntero de acceso a lista descendente.

Definimos una variable de recorrido p y le asignamos el valor que recibe, en este caso l.

Luego hacemos un ciclo while que se recorra mientras p sea distinto de NULL. Dentro del ciclo hacemos un cout mostrando p->dato y vamos moviendo p pero esta vez movemos de anterior en anterior, en vez de mover al siguiente.

```
void mostrarDesc(Nodo*l)
{
    Nodo*p=l;
    while(p!=NULL)
    {
        cout<<p->dato<<endl;
        p=p->ant;
    }
}
```

Insertar en forma ordenada

Para insertar en forma ordenada vamos a usar la función insertar. La misma recibe por referencia los dos punteros a la lista lis1 y lis2 (al primer elemento y al último) y el elemento que quiero insertar.

Lo primero que hacemos es definir dos variables p y r. Con p voy a pedir memoria y luego al dato de p le voy a asignar el dato que quiero insertar.

```
void insertar(Nodo*&lis1, Nodo*&lis2, int nro)
{
    Nodo*p, *r;
    p=new Nodo;
    p->dato=nro;
```

Tenemos 4 casos en los que vamos a insertar en una lista.

- **Insertar a una lista vacía (no hay antecesor ni sucesor):** en este caso tenemos que preguntar si lis1 y lis2 son iguales a NULL. Lo que hago acá es ponerle al siguiente de p y al anterior de p NULL y a lis1 y lis2 los hago igual a p para que apunten a lo mismo.

```
if(lis1==NULL && lis2==NULL) //p no va a tener ni sucesor ni antecesor (agrega a lista vacía)
{
    p->sig=p->ant=NULL;
    lis1=lis2=p;
}
```

Si la lista no está vacía, entonces inicializo r en el puntero al primer elemento de la lista (lis1). Luego de hacemos un ciclo while mientras r sea distinto de NULL y el dato de r sea menor al dato que quiero ingresar. En este caso lo que va a ocurrir es que quiero agregar un dato mayor al primer elemento y entonces tengo que ubicar al sucesor de dicho dato.

```

else
{
    r=lis1;
    while (r!=NULL&&r->dato<nro)
        r=r->sig;

```

- **Insertar al principio de la lista (el dato va a tener sucesor y no antecesor):** en este caso preguntamos si r es igual a lis1, el puntero al primero. Si es así, al anterior de p le ponemos NULL y al siguiente de p le ponemos r. Al anterior de r, es decir, al que era el primer elemento antes de insertar el nuevo, le ponemos p. Finalmente, lis1, que apuntaba al menor de todos, va a dejar de apuntar a ese para apuntar al nuevo elemento que agregamos, entonces lis1=p.

```

if(r==lis1) //p va a tener sucesor y no antecesor
{
    p->ant=NULL;
    p->sig=r;
    r->ant=p;
    lis1=p;
}

```

- **Insertar al final de la lista (el dato va a tener antecesor pero no sucesor):** en este caso preguntamos si r es igual a NULL, pues el siguiente debe ser NULL ya que no tiene sucesor. Si es así, al anterior de p le ponemos lis2 (el puntero al último elemento de la lista) y al siguiente de p le pongo r. Al siguiente de lis2 le pongo p, o sea, al siguiente del elemento que antes era el último hago que apunte al que agregué al final. Finalmente, lis2 va a ser igual a p, pues debe apuntar al nuevo último elemento.

```

else
{
    if(r==NULL) //p va a tener antecesor y no sucesor
    {
        p->ant=lis2;
        p->sig=r;
        lis2->sig=p;
        lis2=p;
    }
}

```

- **Insertar un elemento intermedio (el dato va a tener antecesor y sucesor):** acá al anterior de p le asignamos el anterior de r. Recordemos que r va a ser recorrido hasta el elemento posterior al elemento que vamos a insertar. Al siguiente de p le asignamos r. Ahora para corregir los lazos tenemos dos opciones:
 - La primera usar una variable intermedia llamada anterior a la que le asignamos el anterior de r (o sea le asignamos el nodo anterior) y luego al siguiente del nodo anterior lo cambio para que en lugar de que apunte a lo que estaba apuntando, apunte a p. Luego hago que el anterior de r apunte a p.

- La otra forma es hacer directamente $r \rightarrow \text{ant} \rightarrow \text{sig} = p$. Es decir, el miembro siguiente del nodo apuntado por el miembro anterior de r . Luego hago que el anterior de r apunte a p .

```
else
{
    //p va a tener sucesor y antecesor
    p->ant=r->ant;
    p->sig=r;
    //Nodo*anterior=r->ant;
    //anterior->sig=p;
    r->ant->sig=p;
    r->ant=p;
}
```

El código total queda así:

```
void insertar(Nodo*&lis1, Nodo*&lis2, int nro)
{
    Nodo*p, *r;
    p=new Nodo;
    p->dato=nro;
    if(lis1==NULL && lis2==NULL) //p no va a tener ni sucesor ni antecesor (agrega a lista vacia)
    {
        p->sig=p->ant=NULL;
        lis1=lis2=p;
    }
    else
    {
        r=lis1;
        while(r!=NULL && r->dato<nro)
            r=r->sig;

        if(r==lis1) //p va a tener sucesor y no antecesor
        {
            p->ant=NULL;
            p->sig=r;
            r->ant=p;
            lis1=p;
        }
        else
        {
            if(r==NULL) //p va a tener antecesor y no sucesor
            {
                p->ant=lis2;
                p->sig=r;
                lis2->sig=p;
                lis2=p;
            }
        }
    }
}
```

```

else
{
    //p va a tener sucesor y antecesor
    p->ant=r->ant;
    p->sig=r;
    //Nodo*anterior=r->ant;
    //anterior->sig=p;
    r->ant->sig=p;
    r->ant=p;
}
}
}
}

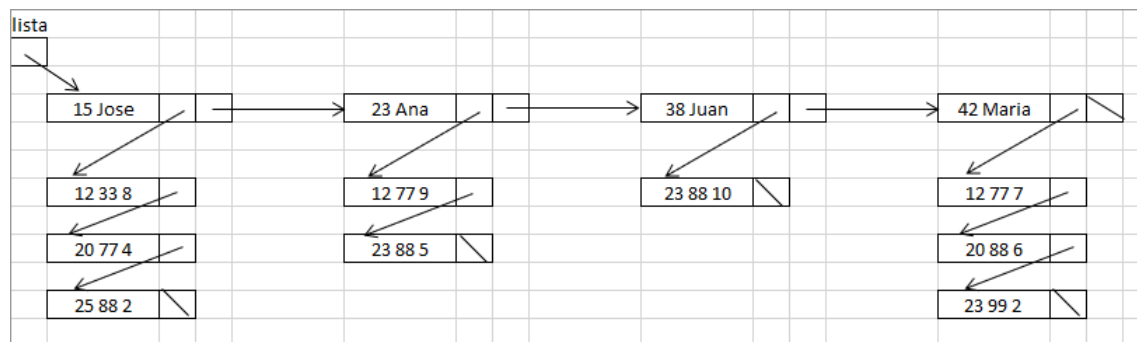
```

Listas y sublistas

La idea es tener listas y, dentro de esas listas, otras listas. En este caso tenemos una lista de alumnos donde, de esa lista, cuelga una lista de exámenes. Cada alumno tiene su lista de exámenes.

Cada nodo de la lista principal, además de tener un puntero al siguiente elemento, va a tener un puntero a una sublista, mientras que la sublistas no. Por ende, el puntero con el que yo recorro la sublista va a ser distinto al puntero con el que recorro la lista principal. Por eso la idea es hacerlo en funciones separadas.

En este caso vamos a sacar el promedio de los exámenes que se encuentran en las sublistas de cada alumno.



Con respecto a los nodos y struct, tenemos un **struct Examen** que va a tener la fecha del examen, el código de la materia y la nota; el **nodo NodoSub** que es el nodo de la sublista que contiene la info del Examen y un siguiente que va a apuntar a un NodoSub; el **struct Alumno** que contiene el DNI, el nombre y un **NodoSub** que contiene la lista con los exámenes, es decir, el puntero de acceso a la sublista de exámenes (El puntero a la sublista es parte de la info de los nodos de la lista principal); por último, tengo el **Nodo de la lista principal** que contiene la info del alumno y el puntero al siguiente.

```

struct Examen
{
    unsigned fecha;
    unsigned codMat;
    unsigned nota;
};
struct NodoSub
{
    Examen info;
    NodoSub*sig;
};
struct Alumno
{
    unsigned dni;
    string nombre;
    NodoSub*listaExam;
};
struct Nodo
{
    Alumno info;
    Nodo*sig;
};

```

Vamos a ingresar por teclado todos los exámenes de todos los alumnos que nos interesan. Por cada examen tengo en qué fecha rindió, DNI y nombre del alumno, qué materia rindió y qué nota tuvo.

En el main inicializamos el puntero a la lista de alumnos en NULL y llamamos a la función generar.

```

int main()
{
    Nodo*listaAlumnos=NULL;
    generar(listaAlumnos);

    return 0;
}

```

Cuando yo ingreso un examen con la **función generar** puede pasar que el mismo sea el primero o no. Si no es el primero, debo buscarlo y agregar el examen. Si el alumno no está debo insertarlo ordenadamente y luego le adjunto la sublista con el examen.

Para lograr todo eso debemos hacer lo siguiente:

Armo un alumno con esos datos, poniéndole el DNI, el nombre y a la lista de exámenes le pongo NULL. Luego defino una variable p y llamo a la función buscarInsertar para buscar a ese alumno y en caso de que no esté lo agrega en dicha variable. Si no estaba, en p tengo el puntero a ese alumno con la lista de exámenes vacías. Entonces armo el examen poniéndole el código de materia, la fecha y la nota y finalmente llamo a la función insertar para insertarlo en la sublista. A esta función insertar le mandamos p->info.listaExamen que es el puntero a la sublista. Si el

alumno estaba p me devuelve el puntero al alumno e igualmente le termino ingresando el examen.

```
void generar(Nodo*&lista)
{
    Alumno al;
    Examen ex;
    Nodo*p;
    unsigned dniAl,codMateria,fechaEx,notaOb;
    string nomAl;
    cout<<"DNI alumno: ";
    cin>>dniAl;
    while (dniAl!=0) //ingresa datos del examen
    {
        cout<<"Nombre alumno: ";
        cin>>nomAl;
        cout<<"Fecha examen: ";
        cin>>fechaEx;
        cout<<"Materia rendida: ";
        cin>>codMateria;
        cout<<"Nota: ";
        cin>>notaOb;

        al.dni=dniAl;
        al.nombre=nomAl;
        al.listaExam=NULL;
        Nodo*p=buscarInsertar(lista,al);
        ex.codMat=codMateria;
        ex.fecha=fechaEx;
        ex.nota=notaOb;
        insertar(p->info.listaExam,ex);

        cout<<"DNI alumno: ";
        cin>>dniAl;
    }
}
```

```
void insertar(NodoSub*&lista,Examen ex) //ordenado por codigo de materia
{
    NodoSub*n,*p,*ant;
    n=new NodoSub;
    n->info=ex;
    p=lista;
    while (p!=NULL && p->info.fecha < ex.fecha)
    {
        ant=p;
        p=p->sig;
    }
    n->sig=p;
    if (p!=lista)
        ant->sig=n;
    else
        lista=n;
}
```



```
Nodo* buscarInsertar(Nodo*&lista,Alumno al)
{
    Nodo*ant,*p=lista;
    while (p!=NULL && p->info.dni<al.dni)
    {
        ant=p;
        p=p->sig;
    }
    if (p!=NULL && al.dni==p->info.dni)
        return p;
    else
    {
        Nodo*n=new Nodo;
        n->info=al;
        n->sig=p;
        if (p!=lista)
            ant->sig=n;
        else
            lista=n;
        return n;
    }
}
```